



Table of Contents

1 Individuelle praktische Arbeit (IPA).....	2
1.1 Barcode-Scanner-Anbindung an das Lagersystem.....	2
1.2 Vorwort.....	2
1.3 1. Informieren.....	3
1.3.1 1.1 Ausgangslage.....	3
1.3.2 1.2 Aufgabenstellung.....	3
1.3.3 1.3 Anforderungen.....	3
1.3.4 1.4 Technologien.....	4
1.4 2. Planen.....	4
1.4.1 2.1 Vorgehen.....	4
1.4.2 2.2 Datenmodell.....	4
1.4.3 2.3 Testkonzept.....	5
1.5 3. Entscheiden.....	5
1.6 4. Realisieren.....	6
1.6.1 4.1 EAN-Validierung.....	6
1.6.2 4.2 Auflösung des Artikels.....	6
1.6.3 4.3 Frontend-Behandlung.....	6
1.6.4 4.4 Fehlerbehandlung.....	7
1.7 5. Kontrollieren.....	7
1.7.1 5.1 Testergebnisse.....	7
1.7.2 5.2 Abnahme.....	8
1.8 6. Auswerten.....	8



1.8.1 6.1 Zielerreichung.....	8
1.8.2 6.2 Reflexion.....	8
1.8.3 6.3 Ausblick.....	8
1.9 Arbeitsjournal (Auszug).....	8
1.9.1 Tag 1 – 21.04.2025.....	8
1.9.2 Tag 4 – 24.04.2025.....	9
1.9.3 Tag 8 – 30.04.2025.....	9
1.10 Quellenverzeichnis.....	9

1 Individuelle praktische Arbeit (IPA)

1.1 Barcode-Scanner-Anbindung an das Lagersystem

Nordlicht Logistik AG Informatiker EFZ, Fachrichtung Applikationsentwicklung

Kandidat: Timo Frei Berufsbildner: Andreas Hubmann Fachvorgesetzter: Martina Reusser
Hauptexperte: R. Baumgartner Nebenexperte: S. Kälin

Ausführungszeitraum: 21. April 2025 – 5. Mai 2025 Abgabe: 5. Mai 2025

1.2 Vorwort

Diese Dokumentation beschreibt meine individuelle praktische Arbeit (IPA), welche ich im Rahmen meiner Ausbildung zum Informatiker EFZ bei der Nordlicht Logistik AG durchgeführt habe. Die Nordlicht Logistik AG betreibt in Pratteln ein Verteilzentrum, über das täglich rund 4'000 Paletten für Kunden aus dem Detailhandel kommissioniert werden.

Im Wareneingang wurden die eintreffenden Artikel bisher manuell erfasst. Die Mitarbeitenden tippten die Artikelnummer und die Menge von Hand in eine Erfassungsmaske. Das war fehleranfällig und langsam. Meine Aufgabe war es, die im Betrieb bereits vorhandenen Honeywell-Handscanner an unser Lagersystem «NoLoWMS» anzubinden, sodass ein Wareneingang durch das Scannen des EAN-13-Barcodes gebucht werden kann.

Ich bedanke mich bei meinem Berufsbildner Andreas Hubmann für die Unterstützung sowie bei den Mitarbeitenden im Wareneingang, die mir geduldig ihren Arbeitsablauf gezeigt haben.



1.3 1. Informieren

1.3.1 1.1 Ausgangslage

Das Lagersystem NoLoWMS ist eine interne Webanwendung (Java Spring Boot, PostgreSQL). Der Wareneingang wird heute über die Maske «WE-Erfassung» abgewickelt. Ein Mitarbeiter liest den Lieferschein, sucht die Artikelnummer und erfasst die Menge. Bei rund 350 Positionen pro Schicht entstehen laut Auswertung des letzten Quartals etwa 2,4 % Fehlbuchungen (falsche Artikelnummer oder Zahlendreher bei der Menge).

Die Honeywell Voyager 1450g Scanner sind im Betrieb bereits vorhanden, werden aber nur an der Kasse des Personalshops verwendet. Sie funktionieren im Keyboard-Wedge-Modus, das heisst, ein gescannter Code wird wie eine Tastatureingabe an das aktive Feld gesendet, gefolgt von einem Carriage Return.

1.3.2 1.2 Aufgabenstellung

Es soll eine Erweiterung der WE-Erfassungsmaske realisiert werden, welche das Scannen eines EAN-13-Barcodes verarbeitet, den zugehörigen Artikel aus der Datenbank auflöst und die Position vorbereitet. Der Mitarbeiter muss anschliessend nur noch die Menge bestätigen. Nicht auflösbare Codes müssen klar signalisiert werden.

1.3.3 1.3 Anforderungen

Funktionale Anforderungen:

- FA01: Das System liest einen gescannten EAN-13-Code aus dem Eingabefeld ein.
- FA02: Das System prüft die Prüfziffer des EAN-13-Codes.
- FA03: Das System löst den Code über die Tabelle artikel_ean in eine interne Artikelnummer auf.
- FA04: Bei erfolgreicher Auflösung wird die Artikelbezeichnung und die zuletzt erfasste Menge angezeigt.
- FA05: Bei unbekanntem Code wird eine Fehlermeldung mit dem gescannten Code angezeigt.
- FA06: Mehrfaches Scannen desselben Codes erhöht die Menge um 1.

Nicht-funktionale Anforderungen:

- NFA01: Die Auflösung eines Codes darf nicht länger als 300 ms dauern.
- NFA02: Die Lösung muss ohne zusätzliche Hardware auskommen (bestehende Scanner im Keyboard-Wedge-Modus).



- NFA03: Der Code muss den Firmenstandards von Nordlicht entsprechen und getestet sein.

1.3.4 1.4 Technologien

- Java 17, Spring Boot 3.1
- PostgreSQL 14
- Thymeleaf für das Frontend, ergänzt mit etwas Vanilla-JavaScript für die Scanner-Behandlung
- JUnit 5 und Mockito für die Tests
- Git / GitLab für die Versionsverwaltung

1.4 2. Planen

1.4.1 2.1 Vorgehen

Ich habe die Arbeit nach IPERKA strukturiert und in Teilaufgaben zerlegt. Für die Zeitplanung habe ich die 40 Stunden auf die Phasen verteilt:

Phase	Geplant	Tätigkeit
Informieren	4 h	Ist-Analyse, Scanner-Verhalten testen
Planen	4 h	Konzept, Datenmodell, Testfälle
Entscheiden	2 h	Variantenentscheid Scanner-Anbindung
Realisieren	22 h	Implementierung inkl. Tests
Kontrollieren	4 h	Systemtest, Abnahme mit Wareneingang
Auswerten	4 h	Reflexion, Dokumentation abschliessen

1.4.2 2.2 Datenmodell

Die Tabelle artikel_ean verknüpft die Artikelnummer mit einem oder mehreren EAN-Codes (ein Artikel kann mehrere Gebindegrößen haben):



artikel_ean

ean VARCHAR(13) PRIMARY KEY

artikel_nr VARCHAR(10) NOT NULL REFERENCES artikel(artikel_nr)

gebinde VARCHAR(20)

1.4.3 2.3 Testkonzept

Ich habe mich für einen testgetriebenen Ansatz (TDD) entschieden. Zentral ist die Klasse EanValidator (Prüfziffernberechnung) und der ScanService (Auflösung). Beide werden mit Unit-Tests abgedeckt. Beispiele geplanter Testfälle:

- gültiger Code 7610827000123 → Prüfziffer korrekt
 - Code mit falscher Prüfziffer → wird abgelehnt
 - bekannter Code → liefert Artikelnummer
 - unbekannter Code → wirft ArticleNotFoundException
-

1.5 3. Entscheiden

Für die Anbindung der Scanner standen zwei Varianten zur Auswahl.

Variante A – Keyboard-Wedge: Der Scanner sendet den Code als Tastatureingabe. Ein verstecktes Eingabefeld fängt die Zeichen ab, das abschliessende Carriage Return löst die Verarbeitung aus.

Variante B – SDK-Anbindung: Direkte Ansteuerung der Scanner über das Honeywell-SDK und einen lokalen Dienst.

Kriterium	Variante A	Variante B
Zusätzliche Hardware/Software	keine	lokaler Dienst nötig
Aufwand	gering	hoch
Wartbarkeit	gut	Abhängigkeit vom SDK
Erfüllt NFA02	ja	nein

Ich habe mich für **Variante A** entschieden, weil sie NFA02 erfüllt, mit der vorhandenen Hardware auskommt und im vorgegebenen Zeitrahmen realistisch umsetzbar ist.

1.6 4. Realisieren

1.6.1 4.1 EAN-Validierung

Als Erstes habe ich die Prüfziffernlogik testgetrieben umgesetzt. Ein EAN-13-Code besteht aus 12 Nutzziffern und einer Prüfziffer. Die Prüfziffer ergibt sich aus der gewichteten Summe (abwechselnd Faktor 1 und 3).

```
public boolean isValid(String ean) {
    if (ean == null || ean.length() != 13 || !ean.matches("\\d{13}")) {
        return false;
    }
    int sum = 0;
    for (int i = 0; i < 12; i++) {
        int digit = ean.charAt(i) - '0';
        sum += (i % 2 == 0) ? digit : digit * 3;
    }
    int check = (10 - (sum % 10)) % 10;
    return check == (ean.charAt(12) - '0');
}
```

Zuerst schrieb ich den Test, der rot war, dann die Implementierung, bis der Test grün war. Anschliessend habe ich um Randfälle ergänzt (leerer String, Buchstaben, zu kurze Codes).

1.6.2 4.2 Auflösung des Artikels

Der ScanService löst einen validierten Code über das Repository auf:

```
public ScanResult resolve(String ean) {
    if (!validator.isValid(ean)) {
        throw new InvalidEanException(ean);
    }
    ArtikelEan mapping = repository.findByEan(ean)
        .orElseThrow(() -> new ArticleNotFoundException(ean));
    return new ScanResult(mapping.getArtikelNr(), mapping.getGebinde());
}
```

1.6.3 4.3 Frontend-Behandlung

Im Thymeleaf-Template habe ich ein fokussiertes, verstecktes Eingabefeld eingebaut. Da der Scanner mit einem Carriage Return abschliesst, konnte ich auf das keydown-Ereignis mit Enter reagieren:

```
scanInput.addEventListener('keydown', function (e) {
    if (e.key === 'Enter') {
```



```
e.preventDefault();
sendScan(scanInput.value.trim());
scanInput.value = "";
}
});
```

Eine Herausforderung war, dass der Fokus nach dem Bestätigen der Menge wieder auf das Scanfeld springen muss, sonst gehen die nächsten Scans ins Leere. Das habe ich mit einem `focus()`-Aufruf nach dem erfolgreichen Buchen gelöst.

1.6.4 4.4 Fehlerbehandlung

Unbekannte Codes werden nicht still verworfen, sondern rot mit dem gescannten Code angezeigt (FA05). Das war ein Wunsch aus dem Wareneingang, damit ein Mitarbeiter den Code notieren und dem Stammdatendienst melden kann.

1.7 5. Kontrollieren

1.7.1 5.1 Testergebnisse

Alle 19 Unit-Tests waren am Ende grün. Die Testabdeckung der Kernklassen `EanValidator` und `ScanService` liegt bei 94 % (gemessen mit JaCoCo).

Anforderung	Erfüllt	Nachweis
FA01	ja	Systemtest, Scan im Testsystem
FA02	ja	Unit-Test <code>isValid</code>
FA03	ja	Unit-Test <code>resolve</code>
FA04	ja	Systemtest
FA05	ja	Systemtest mit unbekanntem Code
FA06	ja	Systemtest, doppelter Scan
NFA01	ja	gemessen 40–70 ms
NFA02	ja	keine Zusatzsoftware
NFA03	ja	Code-Review Berufsbildner



1.7.2 5.2 Abnahme

Am 2. Mai habe ich die Lösung mit zwei Mitarbeitenden aus dem Wareneingang getestet. Sie haben eine Testlieferung mit 30 Positionen gescannt. Es gab keine Fehlbuchung. Eine Rückmeldung war, dass ein kurzer Bestätigungston beim erfolgreichen Scan hilfreich wäre – das habe ich als Verbesserung notiert.

1.8 6. Auswerten

1.8.1 6.1 Zielerreichung

Alle funktionalen und nicht-funktionalen Anforderungen wurden erfüllt. Der Wareneingang kann neu per Scan gebucht werden. Der geschätzte Zeitgewinn liegt bei rund 8 Sekunden pro Position.

1.8.2 6.2 Reflexion

Rückblickend habe ich in der Planung den Aufwand für die Fokus-Behandlung im Frontend unterschätzt. Der reine Backend-Teil war dank TDD gut beherrschbar. Wenn ich die Arbeit nochmals machen würde, würde ich früher mit einem echten Scanner am Testsystem prüfen, statt lange mit simulierten Eingaben zu arbeiten.

Gut gelaufen ist die klare Trennung von Validierung und Auflösung. Dadurch waren die Tests einfach und die Fehlersuche schnell.

1.8.3 6.3 Ausblick

Als Nächstes könnte der Bestätigungston umgesetzt und die Lösung auf den Warenausgang (Kommissionierung) ausgeweitet werden.

1.9 Arbeitsjournal (Auszug)

1.9.1 Tag 1 – 21.04.2025

Am Vormittag habe ich mit Andreas die Aufgabenstellung durchgesprochen und mir im Wareneingang den heutigen Ablauf zeigen lassen. Nachmittags habe ich das Verhalten des Honeywell-Scanners in einem Testformular untersucht: Er sendet den Code als Tastatureingabe mit abschliessendem Enter. Aufgetreten ist das Problem, dass mein Testbrowser den Enter als Formularabsenden interpretiert hat. Gelöst mit `preventDefault()`.



1.9.2 Tag 4 – 24.04.2025

Heute die EanValidator-Klasse testgetrieben fertiggestellt. Erst die Tests für gültige und ungültige Prüfziffern geschrieben, dann die Implementierung. Ein Zahlendreher in meiner Gewichtung (Faktor 1 und 3 vertauscht) hat mich rund eine Stunde gekostet, bis ich per Test gemerkt habe, dass ein bekannter gültiger Code abgelehnt wurde. Danach ScanService begonnen.

1.9.3 Tag 8 – 30.04.2025

Frontend-Integration in die WE-Erfassungsmaske. Das Zurücksetzen des Fokus auf das Scanfeld nach dem Buchen hat länger gedauert als geplant, weil der Fokus nach einem Server-Roundtrip verloren ging. Mit einem gezielten focus() nach der Antwort gelöst. Am Ende des Tages lief ein kompletter Scan-Durchlauf im Testsystem.

1.10 Quellenverzeichnis

- GS1 Switzerland: Aufbau und Prüfziffer EAN-13. <https://www.gs1.ch> (abgerufen 22.04.2025)
- Spring Boot Reference Documentation, Version 3.1. <https://docs.spring.io> (abgerufen 23.04.2025)
- Honeywell Voyager 1450g User Guide, Keyboard-Wedge-Konfiguration (internes PDF)
- Baeldung: Testing with JUnit 5 and Mockito. <https://www.baeldung.com> (abgerufen 25.04.2025)
- Interne Systemdokumentation NoLoWMS, Stand 2024 (Confluence)